Object-Oriented Programming in Java

# Java Generics

## Solutions

Solution code for exercises 6.1–6.8, compiled and run on OpenJDK 25; every output matches the sample run in the exercise sheet, and the mock exam comes with exam-style JUnit 5 tests. At the end: the lines of the Generics slides that do not compile, or are wrong, in Java 25.

# Contents

**About these solutions** **2**

**Part 6 – Generics** **2**
6.1 Generic pairs . . . . . 2
6.2 Generic methods on arrays . . . . . 5
6.3 Your own generic list . . . . . 8
6.4 Your own sorted registry . . . . . 12
6.5 Generic functional helpers . . . . . 16
6.6 Comparator factories, by hand and from the library . . . . . 19
6.7 Wildcards and erasure . . . . . 22
6.8 Mock exam: a library . . . . . 24

**Slide errata (J05)** **37**

# About these solutions

Every solution below was compiled and run on OpenJDK 25 (the tests of 6.8 with JUnit 5), and its output matches the sample run in the exercise. Every file of a solution is shown on its own, imports included; only the `package` line is omitted, since the package is given at the top of each solution. `SimpleList` (6.3) and `Registry` (6.4) are imported by the later exercises, not copied. Each solution ends with a few notes on its key decisions and traps. At the end, the lines of the Generics slides that do not compile or are wrong in Java 25. The runnable project, with Parts 1–6 together, is in `java-exercises-src.zip`, where `run-all.sh` builds and runs everything.

# Part 6 – Generics

## 6.1 Generic pairs

Package `ex6_1`.

File `Couple.java`:

```java
import java.util.Objects;

// Two type parameters: the elements may have different types
public class Couple<A, B> {

    private final A first;
    private final B second;

    public Couple(A first, B second) {
        this.first = first;
        this.second = second;
    }

    public A first() {
        return first;
    }

    public B second() {
        return second;
    }

    // the type arguments are exchanged too
    public Couple<B, A> swap() {
        return new Couple<>(second, first);
    }

    // a generic method of a generic class: C is chosen at each call, A comes from the object
    public <C> Couple<A, C> withSecond(C value) {
        return new Couple<>(first, value);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        // Couple<?, ?>: after erasure the type arguments cannot be checked at run time
        if (!(o instanceof Couple<?, ?> other)) {
            return false;
        }
```

```
        return Objects.equals(first, other.first) && Objects.equals(second, other.second);
    }

    @Override
    public int hashCode() {
        return Objects.hash(first, second);
    }

    @Override
    public String toString() {
        return "<" + first + ", " + second + ">";
    }
}
```

File `Pair.java`:

```
import java.util.Objects;

// Exercise 6.1 - the slides' generic class Pair<T>: two elements of the same type T
public class Pair<T> {

    private T a;
    private T b;

    public Pair(T a, T b) {
        this.a = a;
        this.b = b;
    }

    // A static method cannot use the class's T (it belongs to an instance):
    // it declares its own type parameter, inferred from the arguments
    public static <T> Pair<T> of(T a, T b) {
        return new Pair<>(a, b);
    }

    public T first() {
        return a;
    }

    public T second() {
        return b;
    }

    public void set1st(T x) {
        a = x;
    }

    public void set2nd(T x) {
        b = x;
    }

    // Objects.equals handles null elements (a.equals(x) would throw NullPointerException)
    public boolean contains(T x) {
        return Objects.equals(a, x) || Objects.equals(b, x);
    }

    public Pair<T> swapped() {
        return new Pair<>(b, a);
    }

    @Override
    public String toString() {
```

```
        return "(" + a + ", " + b + ")";
    }
}
```

File `Scores.java`:

```
// A method returns one value: a pair (or a couple) lets it return two,
// with no casts for the caller
public class Scores {

    private Scores() {      // only static methods: no instances
    }

    public static Couple<String, Integer> best(String[] names, int[] scores) {
        if (names.length == 0 || names.length != scores.length) {
            throw new IllegalArgumentException("arrays empty or of different lengths");
        }
        int best = 0;
        for (int i = 1; i < scores.length; i++) {
            if (scores[i] > scores[best]) {      // > and not >=: the first one wins a tie
                best = i;
            }
        }
        return new Couple<>(names[best], scores[best]);   // the int is autoboxed to Integer
    }

    public static Pair<Integer> minMax(int... values) {
        if (values.length == 0) {
            throw new IllegalArgumentException("no values");
        }
        int min = values[0];
        int max = values[0];
        for (int v : values) {
            if (v < min) {
                min = v;
            }
            if (v > max) {
                max = v;
            }
        }
        return new Pair<>(min, max);
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) {
        Pair<String> match = Pair.of("Juventus", "Torino");     // home team, away team
        Pair<Integer> result = new Pair<>(2, 1);                // home goals, away goals
        String away = match.second();                           // no cast
        int homeGoals = result.first();                         // auto-unboxing
        IO.println(match + " " + away + " " + homeGoals + " " + match.swapped() + " "
                + match.contains("Juventus") + " " + result.contains(3));
        result.set1st(3);                                       // the referee's report: 3-1
        IO.println(result + " " + (result.first() + result.second()));
        Couple<String, Integer> entry = new Couple<>("Juventus", 42);
        Couple<Integer, String> byPoints = entry.swap();
        Couple<String, Double> average = entry.withSecond(2.5);
        IO.println(entry + " " + byPoints + " " + average + " "
                + entry.equals(new Couple<>("Juventus", 42)));
        IO.println(Scores.best(new String[]{"Milan", "Inter", "Roma"}, new int[]{7, 9, 9}) + " "
                + Scores.minMax(5, -2, 8, 3));
    }
}
```

`Pair<T>` keeps the slides' accessors and adds the factory `of`, which declares its own `<T>` because a static method cannot use the type parameter of the class (it belongs to an instance); `contains` uses `Objects.equals`, so null elements do not crash. `Couple<A, B>` shows two independent type parameters: `swap()` exchanges the type arguments too (`Couple<B, A>`), and `withSecond` is a generic method of a generic class, whose `C` is chosen at each call while `A` comes from the object. `equals` can only test `instanceof Couple<?, ?>`, because after erasure the type arguments do not exist at run time, so it compares the two elements with `Objects.equals` and `hashCode` uses the same two fields. In `Scores` the primitive results are autoboxed into the pairs (a type argument can never be `int`), and `best` replaces the best only on a strictly greater score, so Inter beats Roma on the tie at 9.

## 6.2 Generic methods on arrays

Package `ex6_2`.

File `Product.java`:

```
// natural order: by price (Double.compare, never a subtraction of doubles cast to int)
public record Product(String name, double price) implements Comparable<Product> {

    @Override
    public int compareTo(Product other) {
        return Double.compare(price, other.price);
    }
}
```

File `ArrayUtils.java`:

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.Objects;
import java.util.function.Predicate;

// Exercise 6.2 - generic methods: the type parameter is declared before the return type
// and inferred by the compiler at each call, so the caller never needs a cast
public class ArrayUtils {

    private ArrayUtils() {      // only static methods: no instances
```

```
    }

    public static <T> int indexOf(T[] a, T x) {
        for (int i = 0; i < a.length; i++) {
            if (Objects.equals(a[i], x)) {      // null-safe, both for x and for the elements
                return i;
            }
        }
        return -1;
    }

    // the slides' method
    public static <T> T remove(T[] a, T el) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] != null && a[i].equals(el)) {
                T removed = a[i];
                a[i] = null;
                return removed;
            }
        }
        return null;
    }

    public static <T> void swap(T[] a, int i, int j) {
        T tmp = a[i];
        a[i] = a[j];
        a[j] = tmp;
    }

    // The bound makes compareTo available on T. Comparable<? super T> (not Comparable<T>)
    // also accepts a T that inherits compareTo from a superclass.
    // The slides declare max void, but it returns a value: the return type is T.
    public static <T extends Comparable<? super T>> T max(T[] a) {
        T max = null;
        for (T current : a) {
            if (current != null && (max == null || max.compareTo(current) < 0)) {
                max = current;
            }
        }
        return max;
    }

    // Comparator<? super T>: a comparator of any supertype of T can compare two T's;
    // null elements are skipped as in the other max, since null means "no maximum yet"
    public static <T> T max(T[] a, Comparator<? super T> order) {
        T max = null;
        for (T current : a) {
            if (current != null && (max == null || order.compare(max, current) < 0)) {
                max = current;
            }
        }
        return max;
    }

    // new T[a.length + 1] does not compile (T is erased): Arrays.copyOf creates
    // the new array with the same run-time type as a
    public static <T> T[] append(T[] a, T x) {
        T[] result = Arrays.copyOf(a, a.length + 1);
        result[a.length] = x;
        return result;
    }

    public static <T> int count(T[] a, Predicate<? super T> cond) {
```

```
        int n = 0;
        for (T x : a) {
            if (cond.test(x)) {
                n++;
            }
        }
        return n;
    }

    public static <T> T[] filter(T[] a, Predicate<? super T> cond) {
        T[] result = Arrays.copyOf(a, a.length);    // big enough, and of the right type
        int n = 0;
        for (T x : a) {
            if (cond.test(x)) {
                result[n++] = x;
            }
        }
        return Arrays.copyOf(result, n);            // trimmed to the elements kept
    }
}
```

File `Main.java`:

```
import java.util.Arrays;
import java.util.Comparator;

public class Main {
    public static void main(String[] args) {
        String[] tags = {"office", "school", "art", "school"};
        Integer[] sold = {13, 8, 21, 5};                    // pieces sold, Monday to Thursday
        IO.println(ArrayUtils.indexOf(tags, "school") + " " + ArrayUtils.indexOf(sold, 99) + " "
                + ArrayUtils.max(tags) + " " + ArrayUtils.max(sold));
        String removed = ArrayUtils.remove(tags, "school");   // no cast
        IO.println(removed + " " + Arrays.toString(tags));
        ArrayUtils.swap(sold, 0, 3);                        // Monday and Thursday were swapped
        IO.println(Arrays.toString(ArrayUtils.append(sold, 34)) + " "
                + ArrayUtils.count(sold, n -> n > 10));
        Product[] ps = {
            new Product("pen", 1.5), new Product("bag", 25.0), new Product("ink", 4.0)
        };
        IO.println(ArrayUtils.max(ps) + " "
                + ArrayUtils.max(ps, Comparator.comparing(Product::name)));
        Product[] cheap = ArrayUtils.filter(ps, p -> p.price() < 5);
        IO.println(Arrays.toString(cheap) + " " + cheap.getClass().getSimpleName());
        Comparator<Object> byText = Comparator.comparing(Object::toString);
        Integer m = ArrayUtils.max(sold, byText);           // T = Integer: see the notes
        IO.println(m);
        Object[] objects = tags;                            // arrays are covariant...
        try {
            objects[0] = 42;                                // ...so this compiles
        } catch (ArrayStoreException e) {
            IO.println("ArrayStoreException: " + e.getMessage());
        }
    }
}
```

Every method declares its type parameter before the return type, so the compiler infers `T` at each call and `String removed = remove(tags, "school")` needs no cast. The slides' `max` is declared `void` although it returns a value: here it returns `T`, with the bound `T extends Comparable<? super T>` that makes `compareTo` available, and both versions skip nulls because `null` already means "no maximum yet". The wildcard of `max(T[], Comparator<? super T>)` matters when the result keeps its type: `Integer m = max(sold, byText)` compiles with `T = Integer` although `byText` is a `Comparator<Object>`. With a plain `Comparator<T>` the call would still compile, because arrays are covariant and the compiler would choose `T = Object`, but the result would be an `Object`, and the assignment to `Integer` would not compile. `new T[n]` does not compile because `T` is erased, so `append` and `filter` call `Arrays.copyOf`, which creates the new array with the run-time type of the argument (hence `Product[]` in the demo); `filter` copies the whole array, fills the front and trims it with a second `copyOf`. The trap is array covariance: `ArrayUtils.<Object>append(new String[]{"s"}, 42)` compiles and throws `ArrayStoreException`, the same failure as `objects[0] = 42` at the end of the demo, which generic types avoid by being invariant.

## 6.3 Your own generic list

Package `ex6_3`.

File `SimpleList.java`:

```java
import java.util.Arrays;
import java.util.Comparator;
import java.util.Iterator;
import java.util.NoSuchElementException;
import java.util.Objects;

// Exercise 6.3 - a growable generic list;
// names and meaning of the methods are those of java.util.List
public class SimpleList<E> implements Iterable<E> {

    private static final int INITIAL_CAPACITY = 4;

    // new E[INITIAL_CAPACITY] does not compile:
    // after erasure the JVM would not know which array to create
    private Object[] elements = new Object[INITIAL_CAPACITY];
    private int size = 0;

    // @SafeVarargs: a generic varargs parameter is an E[] created by the caller; it is safe
    // because the method only reads it and never lets it escape
    @SafeVarargs
    public static <E> SimpleList<E> of(E... items) {
        SimpleList<E> list = new SimpleList<>();
        for (E item : items) {
            list.add(item);
        }
        return list;
    }

    // the only place where an Object becomes an E; the cast is safe:
    // only add and set write the array, and they accept only E's
    @SuppressWarnings("unchecked")
    private E elementAt(int index) {
        return (E) elements[index];
    }

    private void checkIndex(int index, int limit) {
        if (index < 0 || index >= limit) {
            throw new IndexOutOfBoundsException("index " + index + ", size " + size);
        }
    }

    private void growIfFull() {
        if (size == elements.length) {
            elements = Arrays.copyOf(elements, 2 * elements.length);
        }
    }

    int capacity() {        // package-private: only for the demo
        return elements.length;
    }

    public boolean add(E e) {
        growIfFull();
        elements[size++] = e;
        return true;        // as List.add: the list always changes
    }

    public void add(int index, E e) {
        checkIndex(index, size + 1);        // index == size is allowed: it appends
        growIfFull();
        for (int i = size; i > index; i--) {
```

```
            elements[i] = elements[i - 1];  // shift right, starting from the end
        }
        elements[index] = e;
        size++;
    }

    public E get(int index) {
        checkIndex(index, size);
        return elementAt(index);
    }

    public E set(int index, E e) {
        checkIndex(index, size);
        E old = elementAt(index);
        elements[index] = e;
        return old;
    }

    // On a SimpleList<Integer>, remove(1) calls this method (exact match on int),
    // remove(Integer.valueOf(1)) the next one: the same trap as java.util.List
    public E remove(int index) {
        checkIndex(index, size);
        E old = elementAt(index);
        for (int i = index; i < size - 1; i++) {
            elements[i] = elements[i + 1];  // shift left
        }
        // no stale reference: the object can be garbage collected
        elements[--size] = null;
        return old;
    }

    // Object, not E, as in List: asking for any object is type-safe, it simply is not there
    public boolean remove(Object o) {
        int i = indexOf(o);
        if (i < 0) {
            return false;
        }
        remove(i);
        return true;
    }

    public int size() {
        return size;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean contains(Object o) {
        return indexOf(o) >= 0;
    }

    public int indexOf(Object o) {
        for (int i = 0; i < size; i++) {
            if (Objects.equals(o, elements[i])) {
                return i;
            }
        }
        return -1;
    }

    public void clear() {
```

```
        Arrays.fill(elements, 0, size, null);   // the capacity stays, as in ArrayList
        size = 0;
    }

    // As List.sort: stable, and a null comparator means natural order
    public void sort(Comparator<? super E> order) {
        // Arrays.sort needs an E[] to accept a Comparator<? super E>. The cast cannot fail:
        // after erasure E[] is Object[]. The view stays inside this method.
        @SuppressWarnings("unchecked")
        E[] view = (E[]) elements;
        Arrays.sort(view, 0, size, order);
    }

    @Override
    public Iterator<E> iterator() {
        return new SimpleListIterator();
    }

    // An inner (non-static) class: each iterator belongs to a list and reads its elements
    // and size; it uses the E of that list, so it declares no type parameter of its own
    private class SimpleListIterator implements Iterator<E> {
        private int next = 0;       // position of the element next() returns

        @Override
        public boolean hasNext() {
            return next < size;
        }

        @Override
        public E next() {
            if (!hasNext()) {
                throw new NoSuchElementException();
            }
            return elementAt(next++);
        }
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("[");
        for (int i = 0; i < size; i++) {
            if (i > 0) {
                sb.append(", ");
            }
            sb.append(elements[i]);
        }
        return sb.append("]").toString();
    }
}
```

File `Main.java`:

```
import java.util.Comparator;

public class Main {
    public static void main(String[] args) {
        SimpleList<String> names = new SimpleList<>();
        names.add("Zoe");
        names.add("Ann");
        names.add("Bob");
        names.add(1, "Eva");
        IO.println(names + " size=" + names.size() + " " + names.get(1) + " "
```

```
                + names.indexOf("Bob") + " " + names.contains("Max"));
        for (int i = 0; i < 3; i++) {
            names.add("X" + i);
        }
        IO.println(names.size() + " " + names.capacity());
        IO.println(names.remove(0) + " " + names.remove("X1") + " " + names.remove("nobody")
                + " " + names);
        names.sort(Comparator.naturalOrder());
        IO.println(names);
        String initials = "";
        for (String n : names) {
            initials += n.charAt(0);
        }
        IO.println(initials);
        SimpleList<Integer> nums = SimpleList.of(5, 3, 8);
        nums.set(0, nums.get(0) * 10);
        int total = 0;
        for (int n : nums) {
            total += n;
        }
        IO.println(nums + " total=" + total);
        try {
            nums.get(3);
        } catch (IndexOutOfBoundsException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

The elements live in an `Object[]` because `new E[4]` does not compile, and the only cast from `Object` to `E` is in one private method, `elementAt`, whose unchecked warning is suppressed with a reason: only `add` and `set` write the array, and they accept only `E`s. `sort` views the array as an `E[]` (a cast that cannot fail, since at run time `E[]` is just `Object[]`) and calls `Arrays.sort` on the first `size` cells, so, like `List.sort`, it is stable and `sort(null)` means natural order. The iterator is a private inner class: being non-static it reads the `elements` and `size` of its list and uses the list's `E` without a type parameter of its own, and its `next()` throws `NoSuchElementException` at the end as the `Iterator` contract requires. As in `java.util.List`, `contains`, `indexOf` and `remove` take an `Object` (asking for any object is type-safe), and on a `SimpleList<Integer>` the call `remove(1)` removes the element at index 1, not the value 1, which needs `remove(Integer.valueOf(1))`. `add(int, E)` accepts `index == size` (it appends), and both removals null the freed cell so the list keeps no reference to a removed object.

## 6.4 Your own sorted registry

Package `ex6_4`.

File `DuplicateKeyException.java`:

```
// Not generic: a generic class cannot extend Throwable, because catch works on the erased type
public class DuplicateKeyException extends Exception {
    public DuplicateKeyException(String message) {
        super(message);
    }
}
```

File `MissingKeyException.java`:

```
public class MissingKeyException extends Exception {
    public MissingKeyException(String message) {
        super(message);
    }
}
```

File `Registry.java`:

```
import ex6_3.SimpleList;

import java.util.Arrays;
import java.util.Objects;

// Exercise 6.4 - a table from keys to values, sorted by key (the role of a TreeMap in the exams).
// Comparable<? super K>: a key may inherit compareTo,
// e.g. LocalDate is a Comparable<ChronoLocalDate>.
public class Registry<K extends Comparable<? super K>, V> {

    // static: an entry needs no link to its registry. So it cannot see the registry's K and V
    // and declares its own type parameters (here with the same names)
    private static class Entry<K, V> {
        private final K key;
        private V value;

        Entry(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private static final int INITIAL_CAPACITY = 4;

    // new Entry<K, V>[...] does not compile (generic array creation): create an array of
    // Entry<?, ?> and cast it. Safe: the array never leaves the class and holds only Entry<K, V>
    @SuppressWarnings("unchecked")
    private Entry<K, V>[] entries = (Entry<K, V>[]) new Entry<?, ?>[INITIAL_CAPACITY];
    private int size = 0;

    // Binary search on the sorted keys: the position of key if present,
    // otherwise -(insertion point) - 1, the same convention as Arrays.binarySearch
    private int find(K key) {
        Objects.requireNonNull(key, "null key");     // as TreeMap: a null key cannot be compared
        int low = 0;
        int high = size - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;        // (low + high) / 2 could overflow
            int cmp = entries[mid].key.compareTo(key);
            if (cmp < 0) {
                low = mid + 1;
            } else if (cmp > 0) {
                high = mid - 1;
            } else {
                return mid;
            }
        }
        return -(low + 1);      // -1 too, so that 0 means "found at 0"
    }

    private void insertAt(int pos, K key, V value) {
        if (size == entries.length) {
            entries = Arrays.copyOf(entries, 2 * entries.length);
```

```
        }
        for (int i = size; i > pos; i--) {
            entries[i] = entries[i - 1];        // shift the following entries right
        }
        entries[pos] = new Entry<>(key, value);
        size++;
    }

    // as Map.put: null for a new key; note that null is also returned when the old value was null
    public V put(K key, V value) {
        int i = find(key);
        if (i >= 0) {
            V old = entries[i].value;
            entries[i].value = value;
            return old;
        }
        insertAt(-i - 1, key, value);
        return null;
    }

    public void add(K key, V value) throws DuplicateKeyException {
        int i = find(key);
        if (i >= 0) {
            throw new DuplicateKeyException("duplicate key " + key);
        }
        insertAt(-i - 1, key, value);
    }

    public V get(K key) {
        int i = find(key);
        return i >= 0 ? entries[i].value : null;
    }

    // as Map.getOrDefault: the default only for a missing key, not for a key mapped to null
    public V getOrDefault(K key, V otherwise) {
        int i = find(key);
        return i >= 0 ? entries[i].value : otherwise;
    }

    public V require(K key) throws MissingKeyException {
        int i = find(key);
        if (i < 0) {
            throw new MissingKeyException("missing key " + key);
        }
        return entries[i].value;
    }

    public boolean containsKey(K key) {
        return find(key) >= 0;
    }

    public V remove(K key) {
        int i = find(key);
        if (i < 0) {
            return null;
        }
        V old = entries[i].value;
        for (int j = i; j < size - 1; j++) {
            entries[j] = entries[j + 1];        // shift left
        }
        entries[--size] = null;
        return old;
    }
```

```
    public int size() {
        return size;
    }

    // new lists: changing them does not change the registry (unlike Map.keySet(), a view)
    public SimpleList<K> keys() {
        SimpleList<K> result = new SimpleList<>();
        for (int i = 0; i < size; i++) {
            result.add(entries[i].key);
        }
        return result;
    }

    public SimpleList<V> values() {
        SimpleList<V> result = new SimpleList<>();
        for (int i = 0; i < size; i++) {
            result.add(entries[i].value);
        }
        return result;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("{");
        for (int i = 0; i < size; i++) {
            if (i > 0) {
                sb.append(", ");
            }
            sb.append(entries[i].key).append('=').append(entries[i].value);
        }
        return sb.append("}").toString();
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws DuplicateKeyException, MissingKeyException {
        Registry<String, Integer> stock = new Registry<>();
        IO.println(stock.put("pen", 10) + " " + stock.put("ink", 4) + " " + stock.put("pen", 12));
        for (String w : "bag pen bag cap bag".split(" ")) {
            stock.put(w, stock.getOrDefault(w, 0) + 1);
        }
        IO.println(stock + " size=" + stock.size());
        IO.println(stock.keys() + " " + stock.values() + " " + stock.get("zip") + " "
                + stock.containsKey("cap"));
        try {
            stock.add("ink", 1);
        } catch (DuplicateKeyException e) {
            IO.println("error: " + e.getMessage());
        }
        try {
            stock.require("zip");
        } catch (MissingKeyException e) {
            IO.println("error: " + e.getMessage());
        }
        Registry<Integer, String> byYear = new Registry<>();
        byYear.put(2017, "Delivery");
        byYear.put(2014, "Trail");
        byYear.put(2016, "Restaurants");
        IO.println(byYear + " " + byYear.remove(2014) + " " + byYear.keys());
    }
}
```

`Entry<K, V>` is a static nested class, so it cannot see the registry's `K` and `V` and declares its own; `new Entry<K, V>[n]` is a generic array creation and does not compile, so the registry creates an `Entry<?, ?>[]` and casts it, the one unchecked warning, safe because the array never leaves the class. `find` is a binary search that returns `-(insertion point) - 1` for a missing key, the convention of `Arrays.binarySearch`, so a single search serves both lookup and insertion, and the insertion shifts the following entries one place to the right. `put` behaves like `Map.put` (it returns the old value, or `null` for a new key, and also for a key that held `null`), while `getOrDefault`, like `Map.getOrDefault`, returns the default only when the key is missing, not when its value is `null`. The bound `K extends Comparable<? super K>` also accepts keys that inherit `compareTo` from a supertype, such as `LocalDate`, which is a `Comparable<ChronoLocalDate>`. The two exceptions cannot be generic, because a generic class may not extend `Throwable`: `catch` would need type arguments that erasure has removed.

## 6.5 Generic functional helpers

Package `ex6_5`.

File `Sale.java`:

```
public record Sale(String product, String category, int qty, double price) {
}
```

File `Fun.java`:

```java
import ex6_3.SimpleList;
import ex6_4.Registry;

import java.util.function.BinaryOperator;
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.Supplier;
import java.util.function.ToDoubleFunction;

// Exercise 6.5 - each method fixes the loop; the behaviour arrives as a functional object.
// The wildcards follow "producer extends, consumer super": a function consumes T's (? super T)
// and produces R's (? extends R), so e.g. a Function<Object, String> can map a SimpleList<Sale>.
public class Fun {

    private Fun() {         // only static methods: no instances
    }

    public static <T, R> SimpleList<R> map(SimpleList<T> list,
            Function<? super T, ? extends R> f) {
        SimpleList<R> result = new SimpleList<>();
        for (T x : list) {
            result.add(f.apply(x));
        }
        return result;
    }

    public static <T> SimpleList<T> filter(SimpleList<T> list, Predicate<? super T> cond) {
        SimpleList<T> result = new SimpleList<>();
        for (T x : list) {
            if (cond.test(x)) {
                result.add(x);
            }
        }
        return result;
    }

    // BinaryOperator<T> both reads and returns T's: no wildcard can fit both roles
    public static <T> T reduce(SimpleList<T> list, T identity, BinaryOperator<T> op) {
        T result = identity;
        for (T x : list) {
            result = op.apply(result, x);
        }
        return result;
    }

    public static <T> void forEach(SimpleList<T> list, Consumer<? super T> action) {
        for (T x : list) {
            action.accept(x);
        }
    }

    // ToDoubleFunction returns a primitive double: no Double objects are created
    public static <T> double sum(SimpleList<T> list, ToDoubleFunction<? super T> value) {
        double total = 0;
        for (T x : list) {
            total += value.applyAsDouble(x);
        }
        return total;
    }
```

```
    // the pattern of the exams' "for each X, the list of Y":
    // look up the group, create it the first time
    public static <T, K extends Comparable<? super K>> Registry<K, SimpleList<T>> groupBy(
            SimpleList<T> list, Function<? super T, ? extends K> key) {
        Registry<K, SimpleList<T>> groups = new Registry<>();
        for (T x : list) {
            K k = key.apply(x);
            SimpleList<T> group = groups.get(k);
            if (group == null) {
                group = new SimpleList<>();
                groups.put(k, group);
            }
            group.add(x);       // group is a reference to the list inside the registry
        }
        return groups;
    }

    public static <T, K extends Comparable<? super K>> Registry<K, Integer> countBy(
            SimpleList<T> list, Function<? super T, ? extends K> key) {
        Registry<K, Integer> counts = new Registry<>();
        for (T x : list) {
            K k = key.apply(x);
            // Integer objects are immutable: put a new one
            counts.put(k, counts.getOrDefault(k, 0) + 1);
        }
        return counts;
    }

    // a Supplier takes nothing: each call of get() gives the next element
    public static <T> SimpleList<T> generate(int n, Supplier<? extends T> next) {
        SimpleList<T> result = new SimpleList<>();
        for (int i = 0; i < n; i++) {
            result.add(next.get());
        }
        return result;
    }
}
```

File `Main.java`:

```
import ex6_3.SimpleList;
import ex6_4.Registry;

import java.util.Locale;
import java.util.function.BiConsumer;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.Supplier;
import java.util.function.UnaryOperator;

public class Main {
    public static void main(String[] args) {
        SimpleList<Sale> sales = SimpleList.of(
                new Sale("pen", "office", 10, 1.5),
                new Sale("ink", "office", 2, 4.0),
                new Sale("apple", "food", 6, 0.5),
                new Sale("bread", "food", 1, 2.25),
                new Sale("bag", "travel", 1, 25.0));
        Predicate<Sale> office = x -> x.category().equals("office");
        Predicate<Sale> big = x -> x.qty() >= 5;
        IO.println(Fun.map(Fun.filter(sales, office.and(big)), Sale::product) + " "
                + Fun.map(Fun.filter(sales, office.negate().or(big)), Sale::product));
```

```
        Function<Sale, Double> amount = x -> x.qty() * x.price();
        Function<Sale, String> label = amount.andThen(a -> String.format(Locale.US, "%.2f", a));
        IO.println(Fun.map(sales, label));
        Function<String, Integer> length = String::length;
        IO.println(Fun.map(sales, length.compose(Sale::product)));
        IO.println(Fun.reduce(Fun.map(sales, Sale::qty), 0, Integer::sum) + " "
                + Fun.sum(sales, x -> x.qty() * x.price()));
        Registry<String, SimpleList<Sale>> byCategory = Fun.groupBy(sales, Sale::category);
        BiConsumer<String, Integer> show = (k, n) -> IO.print(k + ":" + n + " ");
        for (String k : byCategory.keys()) {
            show.accept(k, byCategory.get(k).size());
        }
        IO.println();
        IO.println(Fun.countBy(sales, x -> x.price() < 2 ? "cheap" : "normal"));
        UnaryOperator<String> shout = w -> w.toUpperCase() + "!";
        Fun.forEach(Fun.map(Fun.filter(sales, big), Sale::product),
                p -> IO.print(shout.apply(p) + " "));
        IO.println();
        Supplier<String> codes = new Supplier<>() {     // an object with its own state
            private int last = 0;

            @Override
            public String get() {
                return "S" + (++last);
            }
        };
        IO.println(Fun.generate(3, codes) + " " + codes.get());
    }
}
```

The wildcards follow ”producer extends, consumer super”: the functions only receive `T`s (`? super T`) and their results are only read (`? extends R`, `? extends K`), so the caller may pass more general functions, while `reduce` keeps a plain `BinaryOperator<T>` because its operator both takes and returns `T`. `groupBy` is the pattern of every exam report ”for each X, the list of Y”: get the group, create and `put` it the first time, then add to it; the list returned by `get` is the same object stored in the registry, so no second `put` is needed. `countBy` must instead `put` a new count every time, because an `Integer` cannot be changed. The demo combines predicates with the default methods `and`, `or` and `negate`, chains two functions with `andThen` (first `amount`, then the formatting) and `compose` (first `Sale::product`, then `length`), and uses `ToDoubleFunction` in `sum` to avoid creating a `Double` for every element. `generate` calls `get()` once per element; the demo's `Supplier` is an anonymous class (with the diamond, allowed since Java 9) because it needs a field: a lambda cannot keep a counter of its own, and `codes.get()` after the list gives `S4` because the same object goes on counting.

## 6.6 Comparator factories, by hand and from the library

Package `ex6_6`.

File `Student.java`:

```
public record Student(int id, String last, String first, double average) {
}
```

File `Comparators.java`:

```java
import ex6_3.SimpleList;

import java.util.Comparator;
import java.util.function.Function;
import java.util.function.ToIntFunction;

// Exercise 6.6 - comparator factories:
// methods that return a new Comparator built from their arguments
public class Comparators {

    private Comparators() {     // only static methods: no instances
    }

    // the key getter reads a T (? super T) and returns some subtype of U (? extends U);
    // U only needs to be comparable with itself or with a supertype (? super U)
    public static <T, U extends Comparable<? super U>> Comparator<T> comparing(
            Function<? super T, ? extends U> key) {
        return (a, b) -> key.apply(a).compareTo(key.apply(b));
    }

    // The slides return key(a) - key(b):
    // with large keys the subtraction overflows and changes sign.
    // (Their signature also has a useless U, and ToIntFunction has only one type parameter.)
    public static <T> Comparator<T> comparingInt(ToIntFunction<? super T> key) {
        return (a, b) -> Integer.compare(key.applyAsInt(a), key.applyAsInt(b));
    }

    // the library's equivalent is the default method order.reversed()
    public static <T> Comparator<T> reverse(Comparator<T> order) {
        return (a, b) -> order.compare(b, a);
    }

    // the second criterion only breaks the ties of the first
    public static <T> Comparator<T> then(Comparator<T> first, Comparator<? super T> second) {
        return (a, b) -> {
            int r = first.compare(a, b);
            return r != 0 ? r : second.compare(a, b);
        };
    }

    // sorts a copy, so the original list does not change;
    // the sort is stable, so ties keep their order
    public static <T> SimpleList<T> top(SimpleList<T> list, int n, Comparator<? super T> order) {
        if (n < 0) {
            throw new IllegalArgumentException("negative n: " + n);
        }
        SimpleList<T> copy = new SimpleList<>();
        for (T x : list) {
            copy.add(x);
        }
        copy.sort(order);
        SimpleList<T> result = new SimpleList<>();
        for (int i = 0; i < n && i < copy.size(); i++) {     // fewer than n elements: all of them
            result.add(copy.get(i));
        }
        return result;
    }
}
```

File `Main.java`:

```
import ex6_3.SimpleList;

import java.util.Comparator;

public class Main {

    static <T> SimpleList<T> sorted(SimpleList<T> list, Comparator<? super T> order) {
        SimpleList<T> copy = new SimpleList<>();
        for (T x : list) {
            copy.add(x);
        }
        copy.sort(order);
        return copy;
    }

    static String names(SimpleList<Student> list) {
        String[] firstNames = new String[list.size()];
        for (int i = 0; i < list.size(); i++) {
            firstNames[i] = list.get(i).first();
        }
        return String.join(" ", firstNames);
    }

    public static void main(String[] args) {
        SimpleList<Student> st = SimpleList.of(
                new Student(11, "Rossi", "Carla", 27.5),
                new Student(7, "Rossi", "Alice", 27.5),
                new Student(3, "Bianchi", "Bruno", 29.0),
                new Student(-2_000_000_000, "Neri", "Ugo", 18.0),
                new Student(2_000_000_000, "Verdi", "Eva", 30.0));
        Comparator<Student> mine = Comparators.then(Comparators.comparing(Student::last),
                Comparators.comparing(Student::first));
        Comparator<Student> lib = Comparator.comparing(Student::last)
                .thenComparing(Student::first);
        IO.println(names(sorted(st, mine)) + " | " + names(sorted(st, lib)));
        IO.println(names(sorted(st, Comparators.comparingInt(Student::id))) + " | "
                + names(sorted(st, (a, b) -> a.id() - b.id())));
        IO.println(names(Comparators.top(st, 2,
                Comparators.reverse(Comparator.comparingDouble(Student::average)))));
        IO.println(names(Comparators.top(st, 3, Comparator.comparingDouble(Student::average)
                .reversed().thenComparing(Student::last).thenComparing(Student::first))));
        Comparator<Object> byText = Comparator.comparing(Object::toString);
        // a Comparator<Object> is a Comparator<? super Integer>
        IO.println(Comparators.top(SimpleList.of(10, 9, 100), 3, byText));
        IO.println(names(st));
    }
}
```

`comparing` has the library's signature: the key getter is a `Function<? super T, ? extends U>` and `U extends Comparable<? super U>`, whereas the slides' `comparingInt` has a useless `U`, gives `ToIntFunction` two type arguments and subtracts the keys; here it uses `Integer.compare`. The subtraction overflows when the keys are far apart: for ids -2,000,000,000 and 2,000,000,000 the difference wraps around to a positive number, so the comparator says that Ugo comes after Eva although it puts Ugo before Bruno and Bruno before Eva, and a sort with an inconsistent comparator gives a result that depends on which pairs the algorithm happens to compare. `then` uses the second comparator only to break the ties of the first and accepts it as `Comparator<? super T>`, and `reverse` is the slides' method, which the library offers as the default method `reversed()`. `top` sorts a copy, so the original list keeps its order (the last line of the demo), and the stable sort keeps tied elements in their original order.

## 6.7 Wildcards and erasure

Package ex6_7.

File Lists.java:

```java
import ex6_1.Pair;
import ex6_3.SimpleList;

import java.util.Comparator;
import java.util.NoSuchElementException;
import java.util.function.IntFunction;

// Exercise 6.7 - wildcards: SimpleList<Integer> is NOT a SimpleList<Number>
// (generics are invariant), but it is a SimpleList<? extends Number>,
// a list we can read Numbers from
public class Lists {

    private Lists() {       // only static methods: no instances
    }

    // ? extends Number: read-only use, the elements are at least Numbers
    public static double sum(SimpleList<? extends Number> list) {
        double total = 0;
        for (Number n : list) {
            total += n.doubleValue();
        }
        return total;
    }

    // src produces T's (? extends T), dst consumes them (? super T):
    // "producer extends, consumer super"
    public static <T> void copy(SimpleList<? super T> dst, SimpleList<? extends T> src) {
        int n = src.size();     // read once: copy(list, list) doubles the list instead of looping
        for (int i = 0; i < n; i++) {
            dst.add(src.get(i));
        }
    }

    // safe: the varargs array is only read
    @SafeVarargs
    public static <T> void addAll(SimpleList<? super T> dst, T... items) {
        for (T x : items) {
            dst.add(x);
        }
    }

    // SimpleList<?>: any list; only Object's methods (here toString) are used on its elements
    public static String describe(SimpleList<?> list) {
        return list.size() + " elements: " + list;
    }

    // non-decreasing: equal neighbours are fine
    public static <T extends Comparable<? super T>> boolean isSorted(SimpleList<T> list) {
        for (int i = 1; i < list.size(); i++) {
            if (list.get(i - 1).compareTo(list.get(i)) > 0) {
                return false;
            }
        }
        return true;
    }

    // the signature of Collections.max (which also adds "Object &" to the bound,
```

```
    // to keep the old erasure)
    public static <T extends Comparable<? super T>> T max(SimpleList<? extends T> list) {
        if (list.isEmpty()) {
            throw new NoSuchElementException("empty list");     // as Collections.max
        }
        T best = list.get(0);
        for (T x : list) {
            if (x.compareTo(best) > 0) {
                best = x;
            }
        }
        return best;
    }

    // two bounds, joined by &: compareTo comes from Comparable, doubleValue from Number.
    // The class comes first, and T is erased to it (Number).
    public static <T extends Number & Comparable<? super T>> double range(SimpleList<T> list) {
        if (list.isEmpty()) {
            throw new NoSuchElementException("empty list");
        }
        T min = list.get(0);
        T max = list.get(0);
        for (T x : list) {
            if (x.compareTo(min) < 0) {
                min = x;
            }
            if (x.compareTo(max) > 0) {
                max = x;
            }
        }
        return max.doubleValue() - min.doubleValue();
    }

    // the slides' pair sorting, with the fixed signature: Comparator<? super T>,
    // so a Comparator<Number> can sort a Pair<Integer>
    public static <T> void sortPair(Pair<T> p, Comparator<? super T> order) {
        if (order.compare(p.first(), p.second()) > 0) {
            T tmp = p.first();
            p.set1st(p.second());
            p.set2nd(tmp);
        }
    }

    // new T[list.size()] does not compile: the caller, who knows the real type, creates the array
    public static <T> T[] toArray(SimpleList<T> list, IntFunction<T[]> maker) {
        T[] result = maker.apply(list.size());
        for (int i = 0; i < list.size(); i++) {
            result[i] = list.get(i);
        }
        return result;
    }
}
```

File `Main.java`:

```
import ex6_1.Pair;
import ex6_3.SimpleList;

import java.util.Comparator;

public class Main {
    public static void main(String[] args) {
        SimpleList<Integer> ints = SimpleList.of(3, 1, 2);
        SimpleList<Double> doubles = SimpleList.of(0.5, 0.25);
        IO.println(Lists.sum(ints) + " " + Lists.sum(doubles));
        IO.println(Lists.range(ints) + " " + Lists.range(doubles));
        SimpleList<Number> numbers = new SimpleList<>();
        Lists.copy(numbers, ints);                      // Integer -> Number
        Lists.copy(numbers, doubles);                   // Double  -> Number
        Lists.addAll(numbers, 7, 8.5);
        IO.println(Lists.describe(numbers) + " sum=" + Lists.sum(numbers));
        SimpleList<Object> objects = new SimpleList<>();
        Lists.addAll(objects, "x", 1, 'c');
        IO.println(Lists.describe(objects));
        IO.println(Lists.isSorted(ints) + " " + Lists.isSorted(SimpleList.of("a", "b", "b")) + " "
                + Lists.max(ints));
        Pair<Integer> p = new Pair<>(4, 2);
        Comparator<Number> byValue = (a, b) -> Double.compare(a.doubleValue(), b.doubleValue());
        Lists.sortPair(p, byValue);   // a Comparator<Number> sorts a Pair<Integer>
        IO.println(p);
        String[] arr = Lists.toArray(SimpleList.of("b", "a"), String[]::new);
        IO.println(arr.length + " " + arr[0] + " " + arr.getClass().getSimpleName());
        SimpleList<String> strings = new SimpleList<>();
        IO.println((strings instanceof SimpleList<?>) + " "
                + (strings.getClass() == ints.getClass()));
    }
}
```

Generics are invariant, so a `SimpleList<Integer>` is not a `SimpleList<Number>`: `sum` accepts `SimpleList<? extends Number>`, from which it can only read `Number`s, and `copy` reads from `? extends T` and writes into `? super T`, which is why the same method copies Integers and then Doubles into a `SimpleList<Number>`; it reads `src.size()` once, so `copy(a, a)` doubles `a` instead of chasing its own growing end forever. `range` has two bounds, `T extends Number & Comparable<? super T>`: `compareTo` finds the extremes and `doubleValue` subtracts them; the class bound must come first, and `T` is erased to it. `describe` takes `SimpleList<?>`, a list of unknown type on which only `Object`'s methods are used, and `max` has the signature of `Collections.max` (minus its `Object &`, kept there for the old erasure) and throws `NoSuchElementException` on an empty list as that method does. `sortPair` is the slides' fixed version with `Comparator<? super T>`, so a `Comparator<Number>` sorts a `Pair<Integer>` in place. Since `new T[n]` cannot be written, `toArray` asks the caller for an `IntFunction<T[]>` such as `String[]::new`, the same idea as `Collection.toArray(IntFunction)`. The last demo line shows erasure: at run time there is only one class `SimpleList`, so `instanceof` cannot check a type argument (with `Object o`, `o instanceof SimpleList<String>` does not compile, `o instanceof SimpleList<?>` does) and a list of strings and one of integers have the same `getClass()`.

## 6.8 Mock exam: a library

Package `ex6_8`.

File `LibraryException.java`:

```
public class LibraryException extends Exception {
    public LibraryException(String message) {
        super(message);
    }
}
```

File `LoanStatus.java`:

```
public enum LoanStatus {
    ACTIVE, RETURNED
}
```

File `Book.java`:

```
// package-private: the facade returns codes and strings, so clients never see a Book
class Book {

    private final int number;       // the n of the code Bn, so that B2 comes before B10
    private final String title;
    private final String author;
    private final int year;
    private final int copies;
    private int onLoan = 0;         // copies lent now
    private int timesLent = 0;      // all the loans, returned ones included

    Book(int number, String title, String author, int year, int copies) {
        this.number = number;
        this.title = title;
        this.author = author;
        this.year = year;
        this.copies = copies;
    }

    int getNumber() {
        return number;
    }

    String getCode() {
        return "B" + number;
    }

    String getTitle() {
        return title;
    }

    String getAuthor() {
        return author;
    }

    int getYear() {
        return year;
    }

    int getTimesLent() {
        return timesLent;
    }

    int available() {
        return copies - onLoan;
    }
```

```
    void lendCopy() {
        onLoan++;
        timesLent++;
    }

    void returnCopy() {
        onLoan--;
    }

    @Override
    public String toString() {
        return getCode() + " " + title + " - " + author + " (" + year + ")";
    }
}
```

File `Loan.java`:

```
// a loan keeps references to its book and member, not their codes: no lookups needed later
class Loan {

    private final int id;
    private final Book book;
    private final Member member;
    private LoanStatus status = LoanStatus.ACTIVE;

    Loan(int id, Book book, Member member) {
        this.id = id;
        this.book = book;
        this.member = member;
    }

    int getId() {
        return id;
    }

    Book getBook() {
        return book;
    }

    Member getMember() {
        return member;
    }

    LoanStatus getStatus() {
        return status;
    }

    boolean isActive() {
        return status == LoanStatus.ACTIVE;
    }

    void markReturned() {
        status = LoanStatus.RETURNED;
    }
}
```

File `Member.java`:

```
class Member {

    private final int id;
    private final String name;
    private int activeLoans = 0;

    Member(int id, String name) {
        this.id = id;
        this.name = name;
    }

    int getId() {
        return id;
    }

    String getName() {
        return name;
    }

    int getActiveLoans() {
        return activeLoans;
    }

    void startLoan() {
        activeLoans++;
    }

    void endLoan() {
        activeLoans--;
    }

    @Override
    public String toString() {
        return id + " " + name;
    }
}
```

File `Library.java`:

```
import ex6_3.SimpleList;
import ex6_4.Registry;

import java.util.Comparator;

// Exercise 6.8 - the facade: every operation of the exam goes through this class
public class Library {

    public static final int MAX_ACTIVE_LOANS = 3;

    // the exam's Map<String, Book>, Map<Integer, Member>, Map<Integer, Loan>
    private final Registry<String, Book> books = new Registry<>();
    private final Registry<Integer, Member> members = new Registry<>();
    private final Registry<Integer, Loan> loans = new Registry<>();

    // ----- R1: books

    public String addBook(String title, String author, int year, int copies)
            throws LibraryException {
        if (copies < 1) {
            throw new LibraryException("invalid number of copies: " + copies);
        }
```

```
        for (Book b : books.values()) {
            if (b.getTitle().equals(title) && b.getAuthor().equals(author)) {
                throw new LibraryException("book already present: " + title + " - " + author);
            }
        }
        // all the checks first: a refused book must not use up a code
        Book book = new Book(books.size() + 1, title, author, year, copies);
        books.put(book.getCode(), book);
        return book.getCode();
    }

    private Book book(String code) throws LibraryException {
        Book b = books.get(code);
        if (b == null) {
            throw new LibraryException("unknown book " + code);
        }
        return b;
    }

    public String getBook(String code) throws LibraryException {
        return book(code).toString();
    }

    public SimpleList<String> findByAuthor(String author) {
        SimpleList<Book> found = new SimpleList<>();
        for (Book b : books.values()) {
            if (b.getAuthor().equalsIgnoreCase(author)) {
                found.add(b);
            }
        }
        found.sort(Comparator.comparingInt(Book::getYear).thenComparing(Book::getTitle));
        SimpleList<String> codes = new SimpleList<>();
        for (Book b : found) {
            codes.add(b.getCode());
        }
        return codes;
    }

    // ----- R2: members

    public int addMember(String name) {
        Member m = new Member(members.size() + 1, name);
        members.put(m.getId(), m);          // the int id is autoboxed to an Integer key
        return m.getId();
    }

    private Member member(int id) throws LibraryException {
        Member m = members.get(id);
        if (m == null) {
            throw new LibraryException("unknown member " + id);
        }
        return m;
    }

    public String getMember(int id) throws LibraryException {
        return member(id).toString();
    }

    // ----- R3: loans

    public int lend(String code, int memberId) throws LibraryException {
        Book b = book(code);
        Member m = member(memberId);
```

```
        if (b.available() == 0) {
            throw new LibraryException("no copy of " + code + " available");
        }
        if (m.getActiveLoans() >= MAX_ACTIVE_LOANS) {
            throw new LibraryException("member " + memberId + " already has "
                    + MAX_ACTIVE_LOANS + " active loans");
        }
        Loan loan = new Loan(loans.size() + 1, b, m);
        loans.put(loan.getId(), loan);
        b.lendCopy();
        m.startLoan();
        return loan.getId();
    }

    private Loan loan(int id) throws LibraryException {
        Loan loan = loans.get(id);
        if (loan == null) {
            throw new LibraryException("unknown loan " + id);
        }
        return loan;
    }

    public void giveBack(int loanId) throws LibraryException {
        Loan loan = loan(loanId);
        if (!loan.isActive()) {
            throw new LibraryException("loan " + loanId + " already returned");
        }
        loan.markReturned();
        loan.getBook().returnCopy();
        loan.getMember().endLoan();
    }

    // an unknown code or loan id: the same errors as everywhere else
    public int available(String code) throws LibraryException {
        return book(code).available();
    }

    public LoanStatus status(int loanId) throws LibraryException {
        return loan(loanId).getStatus();
    }

    // ----- R4: reports

    public SimpleList<String> mostLent(int n) {
        SimpleList<Book> lent = new SimpleList<>();
        for (Book b : books.values()) {
            if (b.getTimesLent() > 0) {
                lent.add(b);
            }
        }
        lent.sort(Comparator.comparingInt(Book::getTimesLent).reversed()
                .thenComparingInt(Book::getNumber));
        SimpleList<String> result = new SimpleList<>();
        for (int i = 0; i < n && i < lent.size(); i++) {
            Book b = lent.get(i);
            result.add(b.getCode() + ":" + b.getTimesLent());
        }
        return result;
    }

    public Registry<String, SimpleList<String>> activeLoans() {
        Registry<String, SimpleList<String>> result = new Registry<>();
        for (Loan loan : loans.values()) {
```

```
            if (loan.isActive()) {
                String name = loan.getMember().getName();
                SimpleList<String> titles = result.get(name);
                if (titles == null) {
                    titles = new SimpleList<>();
                    result.put(name, titles);
                }
                titles.add(loan.getBook().getTitle());
            }
        }
        // values() copies the references, not the lists:
        // sorting them sorts the lists in the registry
        for (SimpleList<String> titles : result.values()) {
            titles.sort(Comparator.naturalOrder());
        }
        return result;
    }

    public Registry<Integer, Integer> titlesPerDecade() {
        Registry<Integer, Integer> result = new Registry<>();
        for (Book b : books.values()) {
            // integer division drops the units: 1965 -> 1960
            int decade = b.getYear() / 10 * 10;
            result.put(decade, result.getOrDefault(decade, 0) + 1);
        }
        return result;
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws LibraryException {
        Library lib = new Library();
        String dune = lib.addBook("Dune", "Herbert", 1965, 2);
        String found = lib.addBook("Foundation", "Asimov", 1951, 1);
        String robots = lib.addBook("I, Robot", "Asimov", 1950, 1);
        String neu = lib.addBook("Neuromancer", "Gibson", 1984, 1);
        int mario = lib.addMember("Mario Rossi");
        int anna = lib.addMember("Anna Bianchi");
        IO.println(lib.getBook(dune) + " | " + lib.findByAuthor("asimov") + " | "
                + lib.getMember(anna));
        int l1 = lib.lend(dune, mario);
        int l2 = lib.lend(dune, anna);
        int l3 = lib.lend(found, anna);
        try {
            lib.lend(dune, mario);
        } catch (LibraryException e) {
            IO.println("error: " + e.getMessage());
        }
        lib.giveBack(l1);
        int l4 = lib.lend(dune, mario);
        lib.lend(neu, mario);
        IO.println(l1 + " " + l2 + " " + l3 + " " + l4 + " " + lib.status(l1) + " "
                + lib.status(l4) + " " + lib.available(dune));
        IO.println(lib.mostLent(3));
        IO.println(lib.activeLoans());
        IO.println(lib.titlesPerDecade());
        try {
            lib.giveBack(l1);
        } catch (LibraryException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

The tests (`test/ex6_8`):

File `TestR1_Books.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import ex6_3.SimpleList;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class TestR1_Books {

    private Library lib;
    private String dune;
    private String found;
    private String robots;

    @BeforeEach
    public void setUp() throws LibraryException {
        lib = new Library();
        dune = lib.addBook("Dune", "Herbert", 1965, 2);
        found = lib.addBook("Foundation", "Asimov", 1951, 1);
        robots = lib.addBook("I, Robot", "Asimov", 1950, 1);
    }
```

```
    @Test
    public void testCodes() throws LibraryException {
        assertEquals("B1", dune, "Wrong code for the first book");
        assertEquals("B2", found, "Wrong code for the second book");
        assertEquals("B3", robots, "Wrong code for the third book");
        assertEquals("B4", lib.addBook("Neuromancer", "Gibson", 1984, 1),
                "Wrong code for the fourth book");
    }

    @Test
    public void testGetBook() throws LibraryException {
        assertEquals("B1 Dune - Herbert (1965)", lib.getBook(dune),
                "Wrong description of the book");
        assertEquals("B3 I, Robot - Asimov (1950)", lib.getBook(robots),
                "Wrong description of the book");
        assertThrows(LibraryException.class, () -> lib.getBook("B99"), "Unknown code accepted");
    }

    @Test
    public void testDuplicateBook() throws LibraryException {
        assertThrows(LibraryException.class, () -> lib.addBook("Dune", "Herbert", 1984, 3),
                "Book with the same title and author accepted");
        assertEquals("B4", lib.addBook("Dune", "Anderson", 2007, 1),
                "Same title with another author should be accepted, with the next code");
    }

    @Test
    public void testInvalidCopies() throws LibraryException {
        assertThrows(LibraryException.class, () -> lib.addBook("Neuromancer", "Gibson", 1984, 0),
                "Zero copies accepted");
        assertThrows(LibraryException.class, () -> lib.addBook("Neuromancer", "Gibson", 1984, -1),
                "Negative copies accepted");
        assertEquals("B4", lib.addBook("Neuromancer", "Gibson", 1984, 1),
                "A refused book should not use up a code");
    }

    @Test
    public void testFindByAuthor() {
        SimpleList<String> codes = lib.findByAuthor("Asimov");
        assertEquals(2, codes.size(), "Wrong number of books by Asimov");
        assertEquals("B3", codes.get(0), "Books should be sorted by year");
        assertEquals("B2", codes.get(1), "Books should be sorted by year");
        assertEquals(0, lib.findByAuthor("Tolkien").size(),
                "No books expected for an unknown author");
    }

    @Test
    public void testFindByAuthorIgnoreCase() {
        SimpleList<String> codes = lib.findByAuthor("aSiMoV");
        assertEquals(2, codes.size(), "The author should be compared ignoring case");
        assertEquals("B1", lib.findByAuthor("HERBERT").get(0),
                "The author should be compared ignoring case");
    }

    @Test
    public void testFindByAuthorSameYear() throws LibraryException {
        String currents = lib.addBook("The Currents of Space", "Asimov", 1952, 1);
        String empire = lib.addBook("Foundation and Empire", "Asimov", 1952, 1);
        SimpleList<String> codes = lib.findByAuthor("asimov");
        assertEquals(4, codes.size(), "Wrong number of books by Asimov");
        assertEquals(empire, codes.get(2), "Books of the same year should be sorted by title");
        assertEquals(currents, codes.get(3), "Books of the same year should be sorted by title");
    }
```

```
}
```

File `TestR2_Members.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class TestR2_Members {

    private Library lib;
    private int mario;
    private int anna;

    @BeforeEach
    public void setUp() throws LibraryException {
        lib = new Library();
        lib.addBook("Dune", "Herbert", 1965, 2);
        mario = lib.addMember("Mario Rossi");
        anna = lib.addMember("Anna Bianchi");
    }

    @Test
    public void testIds() {
        assertEquals(1, mario, "Wrong id for the first member");
        assertEquals(2, anna, "Wrong id for the second member");
        assertEquals(3, lib.addMember("Luca Verdi"), "Wrong id for the third member");
    }

    @Test
    public void testGetMember() throws LibraryException {
        assertEquals("1 Mario Rossi", lib.getMember(mario), "Wrong description of the member");
        assertEquals("2 Anna Bianchi", lib.getMember(anna), "Wrong description of the member");
    }

    @Test
    public void testUnknownMember() {
        assertThrows(LibraryException.class, () -> lib.getMember(99), "Unknown id accepted");
        assertThrows(LibraryException.class, () -> lib.getMember(0),
                "Id 0 accepted, ids start from 1");
    }

    @Test
    public void testSameName() throws LibraryException {
        int other = lib.addMember("Mario Rossi");
        assertEquals(3, other, "Two members may have the same name, with different ids");
        assertEquals("3 Mario Rossi", lib.getMember(other), "Wrong description of the member");
    }
}
```

File `TestR3_Loans.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class TestR3_Loans {
```

```
    private Library lib;
    private String dune;
    private String found;
    private String robots;
    private String neu;
    private int mario;
    private int anna;

    @BeforeEach
    public void setUp() throws LibraryException {
        lib = new Library();
        dune = lib.addBook("Dune", "Herbert", 1965, 2);
        found = lib.addBook("Foundation", "Asimov", 1951, 1);
        robots = lib.addBook("I, Robot", "Asimov", 1950, 1);
        neu = lib.addBook("Neuromancer", "Gibson", 1984, 1);
        mario = lib.addMember("Mario Rossi");
        anna = lib.addMember("Anna Bianchi");
    }

    @Test
    public void testLoanIds() throws LibraryException {
        assertEquals(1, lib.lend(dune, mario), "Wrong id for the first loan");
        assertEquals(2, lib.lend(dune, anna), "Wrong id for the second loan");
        assertEquals(3, lib.lend(found, anna), "Wrong id for the third loan");
    }

    @Test
    public void testAvailable() throws LibraryException {
        assertEquals(2, lib.available(dune), "All copies should be available at first");
        int l1 = lib.lend(dune, mario);
        assertEquals(1, lib.available(dune), "A loan should take one copy");
        lib.lend(dune, anna);
        assertEquals(0, lib.available(dune), "No copies should be left");
        lib.giveBack(l1);
        assertEquals(1, lib.available(dune), "A returned copy should be available again");
    }

    @Test
    public void testStatus() throws LibraryException {
        int l1 = lib.lend(robots, mario);
        assertEquals(LoanStatus.ACTIVE, lib.status(l1), "A new loan should be active");
        lib.giveBack(l1);
        assertEquals(LoanStatus.RETURNED, lib.status(l1), "A given back loan should be returned");
    }

    @Test
    public void testUnknownBookOrMember() {
        assertThrows(LibraryException.class, () -> lib.lend("B99", mario),
                "Loan of an unknown book accepted");
        assertThrows(LibraryException.class, () -> lib.lend(dune, 99),
                "Loan to an unknown member accepted");
    }

    @Test
    public void testNoCopyAvailable() throws LibraryException {
        lib.lend(found, mario);
        assertThrows(LibraryException.class, () -> lib.lend(found, anna),
                "Loan without available copies accepted");
        assertEquals(2, lib.lend(dune, anna), "A refused loan should not use up an id");
    }

    @Test
    public void testMaxActiveLoans() throws LibraryException {
```

```
        int l1 = lib.lend(dune, mario);
        lib.lend(found, mario);
        lib.lend(robots, mario);
        assertThrows(LibraryException.class, () -> lib.lend(neu, mario),
                "Fourth active loan accepted");
        assertEquals(4, lib.lend(neu, anna), "Another member should still be able to borrow");
        lib.giveBack(l1);
        assertEquals(5, lib.lend(dune, mario),
                "After a return the member should be able to borrow again");
    }

    @Test
    public void testGiveBackErrors() throws LibraryException {
        assertThrows(LibraryException.class, () -> lib.giveBack(1),
                "Return of an unknown loan accepted");
        int l1 = lib.lend(dune, mario);
        lib.giveBack(l1);
        assertThrows(LibraryException.class, () -> lib.giveBack(l1), "Loan returned twice");
        assertEquals(2, lib.available(dune), "A refused return should not change the copies");
    }
}
```

File `TestR4_Reports.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNull;

import ex6_3.SimpleList;
import ex6_4.Registry;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class TestR4_Reports {

    private Library lib;
    private int annaDune;
    private int annaFoundation;

    @BeforeEach
    public void setUp() throws LibraryException {
        lib = new Library();
        String dune = lib.addBook("Dune", "Herbert", 1965, 2);
        String found = lib.addBook("Foundation", "Asimov", 1951, 1);
        lib.addBook("I, Robot", "Asimov", 1950, 1);
        String neu = lib.addBook("Neuromancer", "Gibson", 1984, 1);
        int mario = lib.addMember("Mario Rossi");
        int anna = lib.addMember("Anna Bianchi");
        int l1 = lib.lend(dune, mario);
        annaDune = lib.lend(dune, anna);
        annaFoundation = lib.lend(found, anna);
        lib.giveBack(l1);
        lib.lend(dune, mario);
        lib.lend(neu, mario);
    }

    @Test
    public void testMostLent() {
        SimpleList<String> top = lib.mostLent(3);
        assertEquals(3, top.size(), "Wrong number of books");
        assertEquals("B1:3", top.get(0), "Returned loans should be counted too");
        assertEquals("B2:1", top.get(1), "Ties should be sorted by code");
        assertEquals("B4:1", top.get(2), "Ties should be sorted by code");
```

```
    }

    @Test
    public void testMostLentLimit() {
        assertEquals(1, lib.mostLent(1).size(), "Only n books expected");
        assertEquals("B1:3", lib.mostLent(1).get(0), "Wrong most lent book");
        assertEquals(3, lib.mostLent(10).size(), "Books never lent should be left out");
    }

    @Test
    public void testActiveLoans() {
        Registry<String, SimpleList<String>> active = lib.activeLoans();
        assertEquals(2, active.size(), "Wrong number of members with active loans");
        assertEquals("Anna Bianchi", active.keys().get(0), "Members should be sorted by name");
        SimpleList<String> titles = active.get("Mario Rossi");
        assertEquals(2, titles.size(), "Wrong number of active loans");
        assertEquals("Dune", titles.get(0), "Titles should be sorted");
        assertEquals("Neuromancer", titles.get(1), "Titles should be sorted");
    }

    @Test
    public void testActiveLoansAfterReturn() throws LibraryException {
        lib.giveBack(annaFoundation);
        lib.giveBack(annaDune);
        Registry<String, SimpleList<String>> active = lib.activeLoans();
        assertEquals(1, active.size(), "Members without active loans should be left out");
        assertNull(active.get("Anna Bianchi"), "Anna Bianchi has no active loans");
    }

    @Test
    public void testTitlesPerDecade() throws LibraryException {
        Registry<Integer, Integer> decades = lib.titlesPerDecade();
        assertEquals(3, decades.size(), "Wrong number of decades");
        assertEquals(1950, decades.keys().get(0), "Decades should be sorted");
        assertEquals(2, decades.get(1950), "Wrong count for the 1950s");
        assertEquals(1, decades.get(1960), "Wrong count for the 1960s");
        assertEquals("{1950=2, 1960=1, 1980=1}", decades.toString(), "Wrong report");
        lib.addBook("Count Zero", "Gibson", 1986, 3);
        assertEquals(2, lib.titlesPerDecade().get(1980), "Books, not copies, should be counted");
    }
}
```

File `AllTests.java`:

```
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectClasses({TestR1_Books.class, TestR2_Members.class, TestR3_Loans.class,
        TestR4_Reports.class})
public class AllTests {
}
```

The facade keeps three registries, `Registry<String, Book>`, `Registry<Integer, Member>` and `Registry<Integer, Loan>`, in place of the exam's maps; a loan holds references to its book and member, and books and members keep counters (copies on loan, times lent, active loans) that only `lend` and `giveBack` change. Every check comes before anything changes, so a refused book or loan uses up no code or id (the demo's fourth loan is number 4). Ties in `mostLent` are broken by the number in the code, as the text asks: B2 comes before B10, which a plain `String` comparison of the codes would get wrong. `activeLoans` is keyed by name, so two members with the same name share one entry, as the text says. In `activeLoans` the titles are grouped first and sorted at the end: `values()` copies the references, not the lists, so sorting them sorts the lists inside the registry. The tests build a new library in `@BeforeEach`, check one point each, declare `throws LibraryException` instead of catching it, and use `assertThrows` for every error the text lists.

## Slide errata (J05)

These lines of the Generics slides do not compile as written, or are wrong or outdated in Java 25; each one was checked with `javac` 25. As in the other decks, code copied from the PDF also brings typographic quotes and dashes (’, “, –), which are illegal characters in Java source.

| Slide | The slide says | Correct Java |
|---|---|---|
| Bounded types | `< T super D >` as the bound of a type parameter | a type parameter can only have `extends` bounds; `super` exists only for wildcards: `? super D` |
| Generic `max` | `public static <T ...> void max(T ary[])` … `return max;` | a `void` method cannot return a value: `public static <T extends Comparable<? super T>> T max(T[] ary)` |
| Generic Comparable | `return this.id – other.id;` | the typographic – is not an operator, and subtraction can overflow: `return Integer.compare(this.id, other.id);` |
| Sorting a pair | `Comparator ci=(i1,i2)-> i1.compareTo(i2);` | with the raw type the parameters are `Object`, which has no `compareTo`: `Comparator<Integer> ci = (i1, i2) -> i1.compareTo(i2);` |
| Generic Comparator | `new Comparator<Student>(){ public void compare(Student a, Student b){ return a.id – b.id }` | `public int compare(Student a, Student b) { return Integer.compare(a.id, b.id); }`, and close the anonymous class with `}` |
| Comparator.comparingInt | `comparingInt(ToIntFunction<T,U> keyGetter)` returning `keyGetter.applyAsInt(a) - keyGetter.applyAsInt(b)` | `ToIntFunction` has one type argument: `static <T> Comparator<T> comparingInt(ToIntFunction<? super T> key)`, comparing with `Integer.compare` (see 6.6) |

| Slide | The slide says | Correct Java |
|---|---|---|
| Comparator.comparing | `comparing(Function<T,U> keyGetter)` | the real parameter is `Function<? super T, ? extends U>`, so that more functions are accepted (see 6.6) |
| Comparator composition | `default <T> Comparator<T> reversed()` and `default <T> Comparator<T> thenComparing(Comparator<T> other)` | they use the interface's own `T`: `default Comparator<T> reversed()`; a new `<T>` would be a different type variable hiding it. The real parameter is `Comparator<? super T> other` |
| Comparator composition | `default <U extends Comparable<U> Comparator<T> thenComparing(Function<T,U> ke)` | a `>` is missing; the real one is `<U extends Comparable<? super U>> Comparator<T> thenComparing(Function<? super T, ? extends U> key)` |
| Type erasure II | `instanceof` cannot be used on generic types | from an `Object`, only `instanceof G<?>` or the raw `G` compiles; since Java 16 `x instanceof G<String>` also compiles when the static type of `x` already guarantees it |
| Comparator composition | `Arrays.sort(sv,(a,b)-> { … }));` | one `)` too many (`error: ';' expected`): `Arrays.sort(sv, (a, b) -> { … });` |
| Constructor reference, Arrays co-variance | `Integer::new`, `i -> new Integer(i)`, `ao[1] = new Integer(1);` | the `Integer(int)` constructor is deprecated since Java 9 (javac warns): use `Integer::valueOf`, `Integer.valueOf(1)`, or just `1` with autoboxing |
| Generic Iterable and Iterator | `Interface java.util.Iterable` | `Iterable` is in `java.lang` (`java.util.Iterable` gives `cannot find symbol`); `Iterator` is in `java.util` |
| Generic method | the callout "element must have same type as array elements" | not checked: `remove(words, 8)` compiles, with `T` inferred as a common supertype of `String` and `Integer`; only assigning the result, as in `String s = remove(words, 8)`, fails |
| Unbounded generic method | the cast version gets "Warning: unchecked cast from T to Comparable" | javac's warning is `unchecked call to compareTo(T) as a member of the raw type Comparable`; the method is still `void` and returns a value (see `max` above) |
| Sort method | "in class Array, sort is defined as ..." | the class is `java.util.Arrays` (`java.lang.reflect.Array` is something else) |
| Functional Interface Composition | `and(Predicate<T> o)`, `or(Predicate<T> o)`, `andThen(Function<R,V> after)`, `compose(Function<V,T> before)` | the library's parameters have wildcards: `Predicate<? super T>`, `Function<? super R, ? extends V>`, `Function<? super V, ? extends T>` |